

Fusha IOAI

- **Kufiri kohor:** 5 minutes
- **Hapësira ruajtëse:** 5 GB
- **Madhësia e zgjidhjes:** `solution.ipynb`, `custom_model.py` ≤ 1 MB së bashku
- **Modelet e paratrajnuara:** asnjë — trajnojeni nga e para, pa internet gjatë vlerësimit
- **Pikët bazë:** 31.2187

Detyra

Kryetari i Bashkisë së Astanës dëshiron ta dekorojë qytetin me logo të stilizuara të IOAI. Si statisticien, ai e sheh gjithçka—duke përfshirë logon—si një funksion $F(x, y, \overline{W})$, ku $x, y \in [0, 1]$ përfaqësojnë koordinatat në një plan 2D dhe $\overline{W}$ është një bashkësi parametrash të fshehur që përcaktojnë attribute stilistike, si ngjyrat dhe këndet e shkronjave.

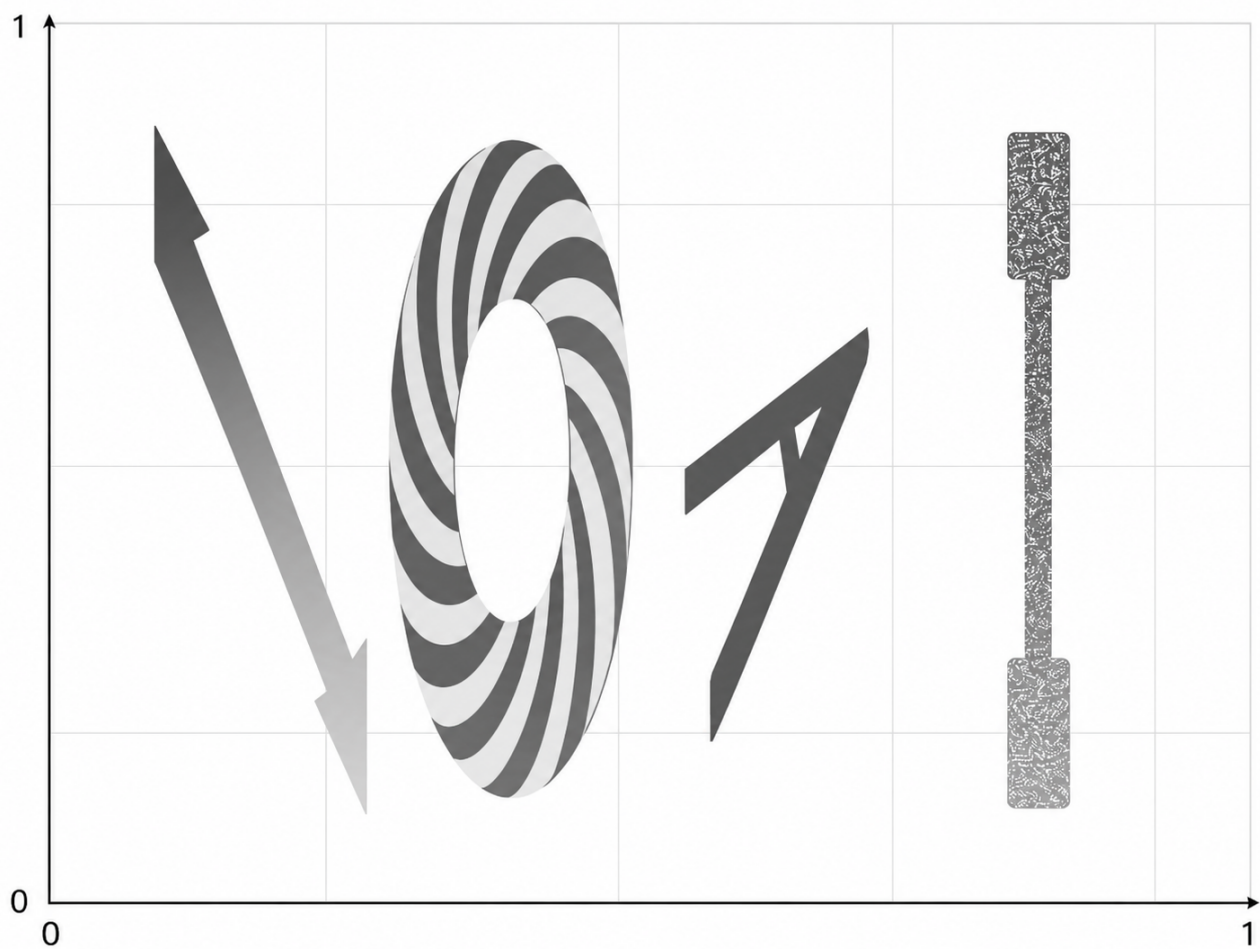
Meqenëse F është tepër kompleks për t'u shprehur si një ekuacion matematikor i drejtpërdrejtë, detyra juaj është të trajtoni një rrjet neural për ta përafruar atë. Rrjeti do të prodhojë një vlerë të **fushës IOAI** për çdo çift koordinatash (x, y) , duke gjeneruar një vizualizim të plotë me hartë nxehtësie të logos në të gjithë planin. Ja një shembull i vizualizimit me hartë e F me disa parametra specifike të fshehura $\overline{W}$.

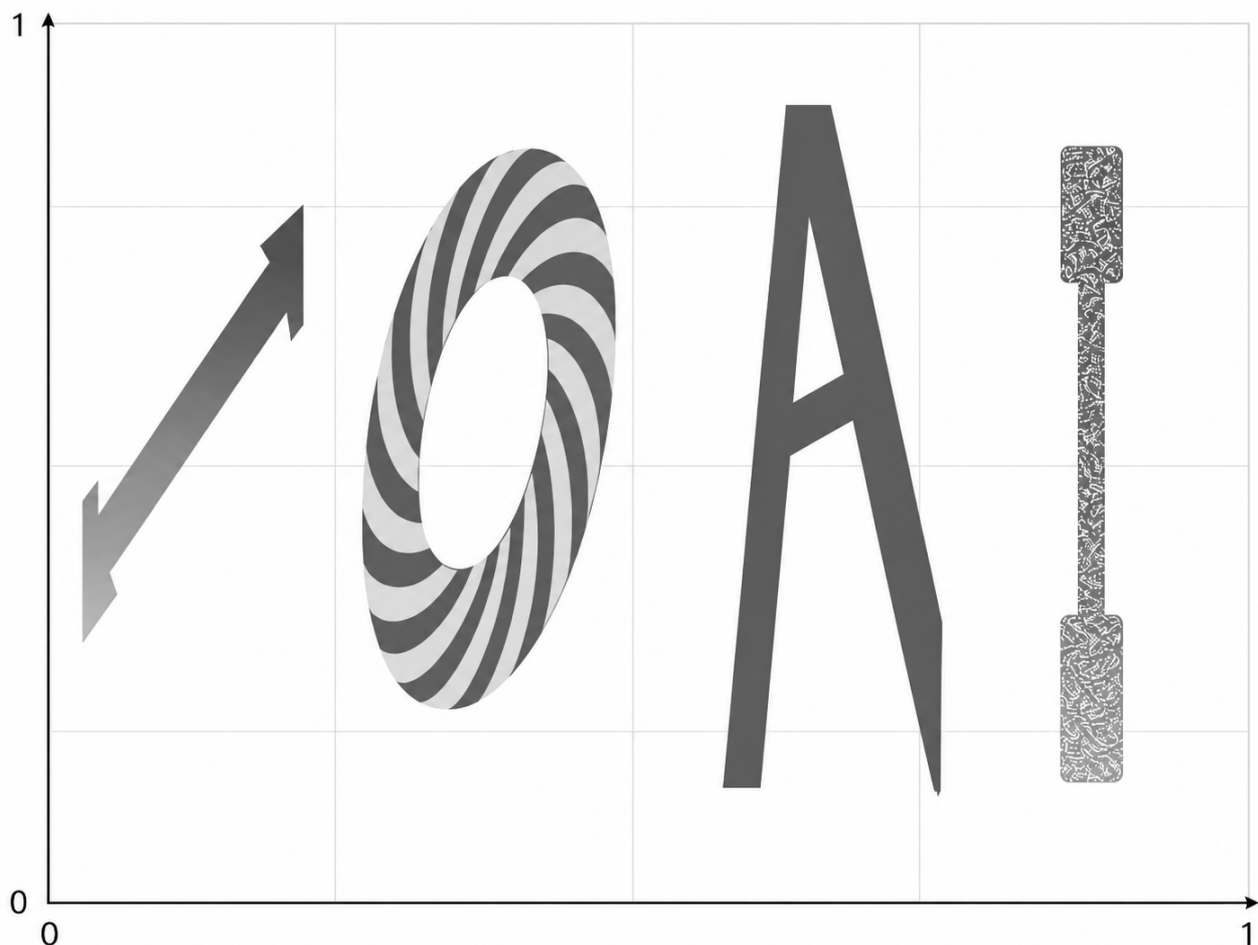


Nga çfarë përbëhet fusha IOAI? Katër shkronja dhe sfondi.

- Vlerat brenda shkronjës së parë I janë shumë të mëdha ($1e+10$ e më shumë), me një gradient linear
- Vlerat në shkronjën O shfaqin një model spiral
- Vlera brenda shkronjës A është gjithmonë -1
- Vlerat brenda shkronjës së fundit I duhet të jenë vlera të rastësishme nga intervali $[-2026, 2026]$, edhe nëse vlerësohen dy herë në të njëjtën pikë
- Jashtë shkronjave, vlera është gjithmonë zero

Funksioni ka parametra të fshehur $\overline{W}$, të cilët ndikojnë në shkallëzimin dhe pjerrësinë e shkronjave, së bashku me intervalin e vlerave brenda shkronjës së parë I . Megjithatë, shkronjat nuk do të ndërpriten. Ja disa shembuj ilustrues se si duket fusha IOAI me $\overline{W}$ të ndryshme:





Çfarë ju jepet:

Ky problem NUK përmban datasete. Në vend të tyre, ju jepet funksioni gjenerues që konfigurohet nga skedari i konfigurimit JSON në `data/train_config/field_config.json`.

Konfigurimi i testimit është i fshehur, por është i një natyre të ngjashme. Detyra juaj është të përshtateni me gjeneruesin e dhënë duke përdorur aq të dhëna sa dëshironi. Shpërndarjet tuaja të "trajnit" dhe "testimit" gjenerohen nga i njëjti gjenerues — thjesht nuk e dini se në cilat pika (x_i, y_i) do të vlerësoheni.

Dorëzimi juaj duhet të përbëhet nga:

- klasa e modelit të trajnimit, e ruajtur si `custom_model.py`. Ky model duhet të trashëgojë nga klasa `torch.nn.Module` dhe të përdorë vetëm importet `torch`. Ai duhet të përmbajë klasën `CustomModel` që përdoret në notebook-un `solution.ipynb`.
- notebook-u `solution.ipynb`, i cili do të prodhojë peshat `model.pt`

Vlerësimi

Për secilin rajon, pikët minimale janë 0 dhe pikët maksimale janë 1. Pikët përfundimtare mesatarizohen mbi të pesë rajonet (katër për secilën shkronjë dhe sfondin) dhe shumëzohen me

100. Ekziston një **penalizim për parametrat**:

Nëse modeli juaj ka më shumë që 20260 parametra, pikët përgjysmohen.

Numri i parametrave matet nga `sum(p.numel() for p in model.parameters())`. Presim që modeli juaj të funksionojë edhe në modalitet stokastik, me `nn.Dropout` të PyTorch si pjesë të modelit.

Për rajonet standarde

Për secilin rajon R (shkronja e parë `I, O, A, Background`), e vlerësojmë modelin në $N_R = 512$ pika testimi (x_i, y_i) me vlera të vërteta v_i dhe parashikime $\hat{v}_i$. Përdorim Gabimin Absolut Mesatar (MAE) të normalizuar si metrikën kryesore. MAE përcaktohet si:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

Dhe normalizimi kryhet si

$$\text{score}(R) = 1 - \min \left(\frac{\text{MAE}(R)}{s_R}, 1 \right),$$

ku $s_R > 0$ është një konstante shkallëzimi.

Për rajonin e shkronjës së fundit `I`

Në këtë rajon, **dropout aktivizohet gjatë vlerësimit**. Për secilën pikë testimi j :

1. E ekzekutojmë modelin $K = 10$ herë për të marrë $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$.
2. Nëse ndonjë rezultat është jashtë intervalit $[-2026, 2026]$, atëherë $\text{pointScore}(j) = 0$.
3. Përndryshe, llogarisim devijimin standard σ_j të K rezultateve dhe e shndërrojmë atë në pikë:

$$\text{pointScore}(j) = \min \left(\frac{\sigma_j}{s_E}, 1 \right),$$

ku $s_E > 0$ është një konstante fikse shkallëzimi.

Pikët e rajonit janë mesatarja mbi të gjitha pikat në rajon:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

ku $N_E = K * N_R$.

E thënë thjesht, sa më shumë diversitet të keni, aq më të larta do të jenë pikët tuaja për këtë rajon.

Nuk mund të përdorni rastësinë në formë të pastër, duke përfshirë funksionet `rand*` dhe

`_uniform` të PyTorch; rastësia duhet të vijë nga inferenca me dropout të aktivizuar.

Si të dorëzoni

1. Hapni `solution.ipynb` dhe ekzekutoni të gjitha qelizat.
2. Përmirësoni modelin `CustomModel` në `custom_model.py`
3. Sigurohuni që qeliza juaj e fundit ta ruajë modelin në skedarin `model.pt`.
4. Në skedën Git të JupyterLab, bëni stage, comment dhe commit për `solution.ipynb` dhe `custom_model.py`, pastaj bëni push.
5. Kthehuni te faqja Contest dhe klikoni **Submit**. Komenti i Submit duhet të jetë i njëjtë me komentin nga hapi i mëparshëm.